Practical Work No. 10
Dynamic Memory Allocation

I. Dynamic Memory Allocation

Dynamic memory allocation involves manually reserving memory space for a variable or an array. Dynamic memory allocation is primarily based on the two methods `malloc` and `free` from the `<stdlib.h>` library:

- **malloc** (….) : Requests the operating system for permission to use memory.
- **free** (….) : Indicates to the OS that the previously allocated memory is no longer needed, allowing it to be reused by other programs.

II. Allocating Memory for a Single Variable

Example:

```
void main() {
    int* P = NULL;
    P = malloc(sizeof(int));
    /* Memory allocation */
    printf("How old are you? ");
    scanf("%d", P);
    printf("You are %d years old\n", *P);
    free(P);  /* Freeing allocated memory */   }
```

III. Allocating Memory for an Array

Memory for an array can be allocated dynamically, and the pointer can be manipulated using arithmetic operations like addition, subtraction, increment, and decrement. Movement is restricted to multiples of the memory size required for the type of the variable being pointed to (`n * sizeof(type)`).

Example:

Solution 1:

```
int n, i;
float *array;
scanf("%d", &n);
/* Reserving memory for n * 4 bytes */
array = (float *)malloc(n * sizeof(float));
/* Filling the array with n real numbers */
printf("\nEnter the %d numbers to sort:\n", n);
for (i = 0; i < n; i++)
    scanf("%f", array + i);
/* Displaying the array */
printf("\nHere are the elements:\n");
for (i = 0; i < n; i++)
    printf("%.2f ", *(array + i));
/* Freeing allocated memory */
free(array);
```

Solution 2:

```
int n;
float *array, *ptr;
scanf("%d", &n);
/* Reserving memory for n * 4 bytes */
array = (float *)malloc(n * sizeof(float));
/* Filling the array with n real numbers */
printf("\n Enter the %d numbers to sort:\n", n);
for (ptr = array; ptr < array + n; ptr++)
    scanf("%f", ptr);
/* Displaying the array */
printf("\n Here are the elements:\n");
for (ptr = array; ptr < array + n; ptr++)
    printf("%.2f ", *ptr);
/* Freeing allocated memory */
free(array);
```

IV. Application Exercises

Exercise 1: Frequency
Given an array `T` containing `n` elements of type `int` and an integer `x`, write a function:

```
int frequency(int *T, int n, int x);
```

This function returns the number of occurrences of `x` in the array `T`.

Exercise 2: Occurrence of 0
Given an array `T` containing `n` elements of type `int`, write a function:

```
void removeZeroOccurrences(int *T, int *n);
```

This function removes all occurrences of the value `0` in the array `T` and shifts the remaining elements.
Note: The number of elements in the array should change.

Exercise 3: Reverse
Write a C program that reads the size `N` of an array `T` of type `int`, fills the array with values entered by the user, and displays the array. Then reverse the elements of the array `T` without using an auxiliary array and display the resulting array.

For this purpose, implement the following functions in C:

```
void fillArray(int *t, int n);
void displayArray(int *t, int n);
void reverseArray(int *t, int n);
```

Remark: Swap the elements of the array using two indices that traverse the array, starting at the beginning and the end, and meet in the middle.